IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2023

DIGITAL ELECTRONICS & COMPUTER ARCHITECTURE

Tuesday 9 May 14:30

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions. Question One carries 40 marks. Question Two and Question Three carry 30 marks each.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : G. Constantinides
Second Marker(s) : T. Clarke

© Imperial College London

DIGITAL ELECTRONICS AND COMPUTER ARCHITECTURE

1. a) Simplify the following Boolean expression. You may use any method, but the method must be clear from your answer.

$$(\overline{A}\,\overline{B}+\overline{C})(\overline{A}\,\overline{B}\,C)$$

[4]

b) Simplify the Boolean expression $Y = \sum 0,2,4,5,10,11$ using a Karnaugh map, leaving your answer in sum-of-products form. Show all your working. (You do not need to consider the elimination of static hazards.)

[4]

c) Rewrite the Boolean expression $A\,\overline{B}\,\overline{C}+\overline{D}$ in minimal product-of-sums form.

[2]

d) Write the decimal values -3 and 10 in 8-bit twos complement. Then also write their respective sign extensions in 16-bit twos complement.

[4]

e) Draw a Mealy state diagram for the circuit shown in Figure 1.1, where the flip-flop's clock inputs are connected to a suitable external clock signal (not shown).

[4]

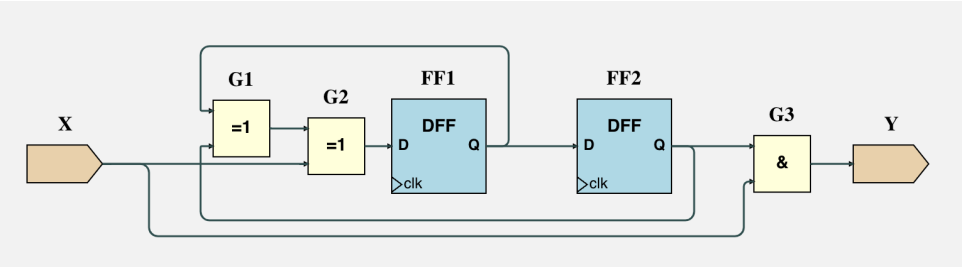


Figure 1.1 A sequential circuit.

f) Write a Boolean sum-of-products equivalent to the expression $B\,\overline{C}+\overline{A}\,C$ but eliminating static hazards.

[2]

```
MOV R0, #255
MOV R1, #10
ADD R1, R1, R1
SBC R1, #2
MOV R2, #0x4E
MOV R3, #0x65
AND R2, R2, R3
LSR R3, R3, #1
XSR R4, R3, #1
MOV R5, #0xFF
LSL R5, R5, #8
ASR R5, R5, #3
```

Figure 1.2 EEP1 program

g) Figure 1.2 shows an EEP1 program which is loaded into the EEP1 instruction ROM `CODEMEM` starting at location 0. The EEP1 CPU executes this program from location 0. Write the values, in hexadecimal, of the 6 registers $R0-R5$, `CODEMEN[0]`, and the EEP1 Program Counter, when the `ASR` instruction is executing.

[8]

h) $X(n-1:0)$ is an n-bit binary value, and $u(X)$, $z(X)$ are respectively the unsigned and twos complement signed interpretations of X. Derive an expression for $u(X)-z(X)$ in terms of the bits of X and prove that it is a multiple of 2^n. Explain what this means for the hardware implementation of twos complement signed arithmetic.

[4]

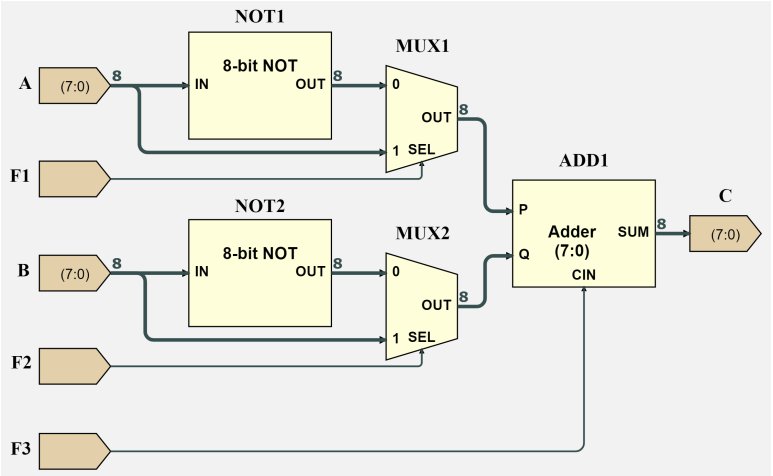


Figure 1.3 EEP1 program

i) Figure 1.3 shows an ALU with data inputs A, B, output C, and control inputs $F1, F2, F3$. Write an algebraic truth table for the ALU with 8 rows, each expressing the output C as an arithmetic expression of the inputs.

[8]

2. You are asked to design a circuit with two Boolean inputs which are to be interpreted in this question as a single two's complement word $X(1:0)$.

The circuit behaves as a two-bit synchronous counter which, on each clock rising edge, adds the current value of its two-bit two's complement state $Y(1:0)$ to the value of its two's complement input $X(1:0)$ to produce its new state. If the new state would overflow in the positive direction, then instead of the sum, the new state becomes the maximum positive value representable as a two-bit two's complement number. Likewise, if the new state would overflow in the negative direction, then instead of the sum, the new state becomes the maximum negative value representable as a two-bit two's complement number.

The output of the circuit is a single bit Z which indicates whether the current state is zero.

An example of this behaviour is shown in the timing diagram of Figure 2.1, for which the starting state is assumed to be $Y(1:0) = 0$.

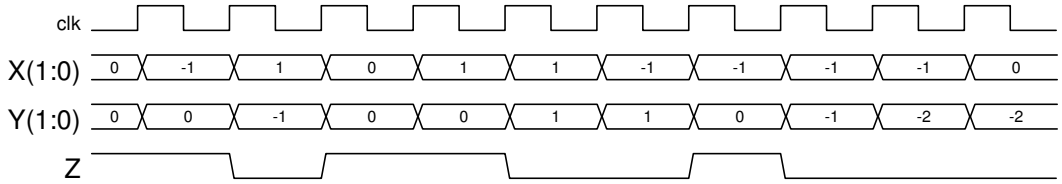


Figure 2.1 Timing diagram.

a) Draw a Moore state diagram of a FSM implementing the required behaviour.

[8]

b) Write out Karnaugh maps for the output, and each bit of the next-state variable, of the FSM. Use the Karnaugh maps to obtain the minimal sum-of-product expressions, explicitly identifying the groups in your Karnaugh maps corresponding to each product term.

[12]

c) Through the application of DeMorgan's Theorem to your sum-of-products expressions, draw a circuit directly implementing this state machine using only D-flip flops and NAND gates. Your NAND gates may have an arbitrary number of inputs. Show all your working.

[10]

3. a) Figure 3.1 shows the required transfer of 4 words of data from memory to registers $R2$-$R5$ in an EEP1 program. Data transfers are indicated by dotted arrows. Write a sequence $P1$ of EEP1 assembler instructions which implements this transfer. Credit will be given for shorter solutions.

[6]

b) State, with reasons for correctness, a single EEP1 instruction that will implement 16-bit bitwise logical NOT: $R0(i) := \overline{R1(i)}$. You may assume the existence of a constant value you specify in a register.

[2]

c) Showing your reasoning, derive an expression for bitwise logical OR, using only bitwise logical AND and NOT operations.

[2]

d) Write in EEP1 assembler a subroutine $P2$ that implements the 16-bit bitwise logical OR operation.

$$R0(i) := R0(i) + R1(i) \quad (+ \text{ is boolean OR}).$$

Credit will be given for correct solutions minimising the number of instructions.

[6]

e) Assume that $P1$ and $P2$ work as specified. Define $x0, x1, x2, x3$ to be the values in memory locations as in Figure 3.1. Write an EEP1 assembler program $P3$ that includes $P1$ and then calls $P2$ multiple times to implement the bitwise operation:

$$R0(i) := x0(i) + x1(i) + x2(i) + x3(i) \quad (+ \text{ is boolean OR}).$$

[4]

f) The code in $P3$ is run on a pipelined implementation of EEP1 with 200MHz clock frequency. The pipeline runs with one stall cycle for each LDR instruction and no other stalls. State the dynamic total of instructions, n, and dynamic number of LDR instructions, m, in the execution of $P3$. Calculate, showing your working, the instruction throughput of $P3$.

[5]

g) Write the entry and exit code necessary to make your program $P3$ into a subroutine complying with the EEP1 Subroutine Call Protocol. Draw the $P3$ stack frame showing the relative position in memory of each stored register.

[5]

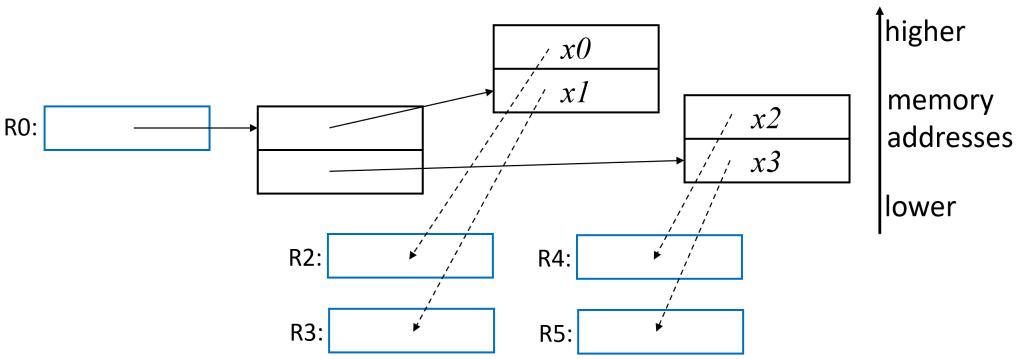


Figure 3.1 Memory pointer diagram